

Contents

Guide 10 — Architecture & Tech Stack, the simple version

1. The idea, in three sentences

2. The three tiers

3. The stack (what, and why)

4. Why SQLite (the question that always comes up)

5. The files

6. Jury questions — ready answers

1

1

1

2

2

2

Guide 10 — Architecture & Tech Stack, the simple version

How Mobile Match Algeria is structured and what it is built with — in plain words, with the reasons behind each choice, and ready answers for the jury.

1. The idea, in three sentences

The application follows a classic **three-tier architecture**: a presentation tier (what the user sees), a logic tier (the rules and endpoints), and a data tier (the database). It is built on one TypeScript framework, **Next.js**, that serves both the interface and the API, with **Prisma + SQLite** for storage. One language (TypeScript) runs end to end.

2. The three tiers

- **Presentation tier** — React components rendered by Next.js (on the server for the first load, on the client for interactivity): the offers grid, the compare page, the recommendation form, the admin dashboard.
- **Logic tier** — Next.js API route handlers under `src/app/api`: they hold the business logic and talk to the database (offers, search, recommendations, auth, notifications).
- **Data tier** — a **SQLite** database accessed **only through Prisma**; the Prisma schema is the single source of truth for the data model.

A request flows: component → API route → Prisma → SQLite → back up to the component.

3. The stack (what, and why)

Layer	Tool	Why
UI + API	Next.js 16 (App Router, Turbopack), React 19 , TypeScript (strict)	one framework for both interface and API; type-safe end to end
Data	Prisma 7 + SQLite	schema as the source of truth, type-safe queries; SQLite is zero-config and portable

Layer	Tool	Why
Auth	NextAuth (Auth.js) v5	maintained, secure session handling
Scraping	Playwright (headless Edge) + Cheerio	a real browser defeats anti-bot layers; Cheerio parses the HTML
Charts	Recharts	simple React charting for the comparison page
Validation	zod	validates API inputs
Tests	Vitest	fast unit tests

4. Why SQLite (the question that always comes up)

The catalogue is small (around 80–100 plans), read-heavy, and runs as a single instance. SQLite fits that perfectly: it is a **single file**, needs **no separate database server**, is fast for this size, and has zero operational overhead. Because everything goes through Prisma, moving to PostgreSQL or MySQL later would be a **configuration change, not a rewrite** — so choosing SQLite now costs nothing in portability.

Likewise **Prisma over raw SQL**: it gives a typed schema, type-checked queries, and migrations, which removes a whole class of runtime errors.

5. The files

- **prisma/schema.prisma** — the data tier (the single source of truth).
- **src/app/api/** — the logic tier (route handlers).
- **src/app/** and **src/components/** — the presentation tier.
- **src/lib/** — shared logic (recommendation, search, scraper, utils, auth).

6. Jury questions — ready answers

- **Why SQLite and not MySQL/PostgreSQL?** The dataset is small and read-heavy on a single instance; SQLite is zero-config and fast enough, and Prisma makes switching databases a config change if the project ever scales.
- **Why Prisma and not raw SQL?** Type-safe queries and a schema-as-source-of-truth catch mistakes at compile time and make migrations clean.
- **Why Next.js for both the UI and the API?** One framework and one language (TypeScript) end to end, with server rendering for fast first loads and API routes for the logic — no separate backend service to maintain.
- **Is it deployable to production?** Yes — the Next.js / Prisma stack runs on any Node.js host; deployment is configuration, identified as a step in the Perspectives chapter.

Companion to the authentication, security, and testing guides.